TEST-CHAMBER: A PORTAL INSPIRED ENVIRONMENT FOR REINFORCEMENT LEARNING

CS748-2022 – FINAL PROJECT REPORT

Ashutosh Sathe
Roll no. – 21q050012
absathe@cse.iitb.ac.in

ABSTRACT

This report introduces "TESTCHAMBER" – a new learning environment for reinforcement learning based on the 3D, physics-based puzzle game "Portal 2". This domain poses a set of new challenges that require agent behavior to be much closer to human thinking process. Despite being a single-player or co-operative 2 player game, Portal 2 requires explicit exploration, memory, dexterity (time sensitive actions) and reasoning in order to solve the puzzles. The report focusses on the *development* of the environment for simulating the game, provides observation, action and possible reward specifications along with a set of open-source Python tools for interfacing with the simulated game. In addition to the simulation, the report also provides ways to dynamically map real human gameplay into set of observations and actions. This capability will enable agents to learn from diverse set of (human generated) solutions presented for a particular puzzle. Experiments suggest that current state-of-the-art models can solve one particular puzzle really well but are unable to generalize *across* puzzles in this environment. Thus TESTCHAMBER offers a new challenging environment for research on better algorithms and model architectures.

1 Introduction

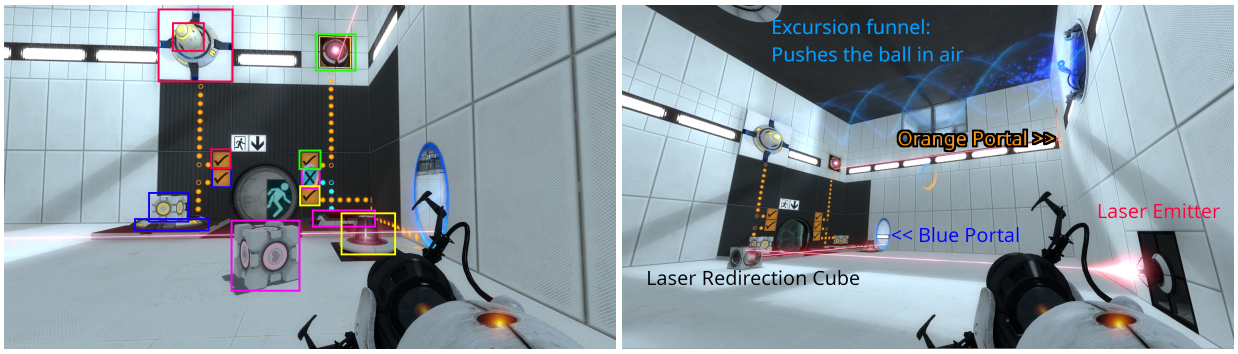
Deep reinforcement learning (RL) has been shown to be incredibly successful by many works such as Mnih et al. [2015], Berner et al. [2019], Vinyals et al. [2019], Pearce and Zhu [2021], Fuchs et al. [2021] in playing and achieving superhuman performance in increasingly difficult computer games. Many of these games (such as Atari games or Gran Turismo) can be argued to be solvable just by having fast sensory response (dexterity). While works on multiagent games such as DOTA 2 agent by Berner et al. [2019], StarCraft II agent by Vinyals et al. [2019] or even Hide-and-Seek agent by Baker et al. [2020] have demonstrated planning capabilities, the environments under consideration had considerable amount of low-level world information. For example, works like Berner et al. [2019] observed the world via 20000 floating point numbers which together perfectly represented the low level information about the world that only game engine has but human player does not.

Moreover, all the previous environments assumed a fixed set of game mechanics and a fixed number of compositions of those mechanics that the agent can use. Therefore, if a new set of game mechanics was added to the environment, the agents may completely fail to adapt to that environment or demonstrate issues like "catastrophic" forgetting where the agent may lose performance on previously learned skills. As opposed to this, if humans were introduced to the new composition of already existing game mechanics via a short tutorial, they can quickly adapt to applying this concept in a novel problem setting. Another crucial aspect of previous works is that they are often adversarial (human(s)-vs-agent(s) or agent(s)-vs-agent(s)), which hinders the ability to directly compare the performance of humans doing the task to performance of agents doing the task. The work on Gran Turismo by Fuchs et al. [2021] game is an exception to this.

The proposed TESTCHAMBER is motivated to address these issues. It presents a fully-featured *Gym* Brockman et al. [2016] interface to a popular single-player or co-operative two-player puzzle based game Portal 2. Agents trained in TESTCHAMBER can be directly compared to human players. The "speedrunning" community for Portal 2 is a group of people that are passionate about trying to complete the main game as fast as possible. There is also a bigger community of "mappers" which create new puzzle maps which are freely available to general players via "Steam Workshop"[1]. This allows a much more mature "human-vs-agent" comparison on the challenging reasoning-heavy puzzle solving task. Since the puzzles are so diverse, the agent must learn a generalizable policy that works *across* various different puzzles.

The report first introduces Portal based puzzle solving and formalizes the expectations from a Portal based RL environment (Sec. 2). These expectations are compared with existing RL environments (Sec. 3). The development

[1] https://steamcommunity.com/workshop/about/?appid=620 – has 800k+ community made puzzles



(a) Complex exit to a test chamber

(b) Full solution to the test chamber

Figure 1: Solving puzzles using portals (best viewed zoomed in)

process of TESTCHAMBER is described in Sec. 4. Finally, the report also demonstrates the TESTCHAMBER environment by training a sample agent and provides discussion about various aspects of the learned agent (Sec. 5, Sec. 6).

2 Solving Puzzles Using Portals

Portal 2[2] is a 3D, first-person, physics based puzzle game made and released by Valve. Each puzzle in Portal 2 is called a TESTCHAMBER. The player spawns at the entrance of this TESTCHAMBER and the only goal is to reach the exit of the chamber. The special aspect of Portal 2 puzzles is the use of "Portal gun" to move the player and other required equipment around the chamber. The portal gun can shoot 2 (colored orange and blue) portals – essentially holes in the walls/floor of the chamber and enables instantaneous bidirectional movement of player or other material between these portals. Also note that *just* reaching the door may not be sufficient to "solve" the puzzle since the exit door may not always be open by default and may require some non-trivial amount of work to open the door.

Figure 1a shows a complex exit dependant on 5 in-game elements – the 3 types (shown in red, blue and pink) of buttons, a "laser relay" (yellow) and a "laser receiver" (green). The connection of these 5 elements to the exit door is shown using "antlines". The "orange" antlines indicates a fulfilled condition while the "blue" antlines indicates the opposite. Figure 1b reveals the complete solution to the chamber. The 2 buttons on the bottom (blue and pink in Figure 1a) are being pressed by 2 cubes. The laser is "redirected" 90 degrees using the "laser redirection cube", activates the "laser relay" (yellow in Figure 1a), passes through the blue portal and comes out of the orange portal to hit a "laser receiver" (green in Figure 1a). The third button on the wall (red in Figure 1a) is being pressed by a cube which is held in air pressed against the wall using an "excursion funnel". Since all 5 of the exit elements are satisfied, the exit door is open and the player can walk through it to complete the chamber. The comprehensive list of all in-game mechanics can be found here[3].

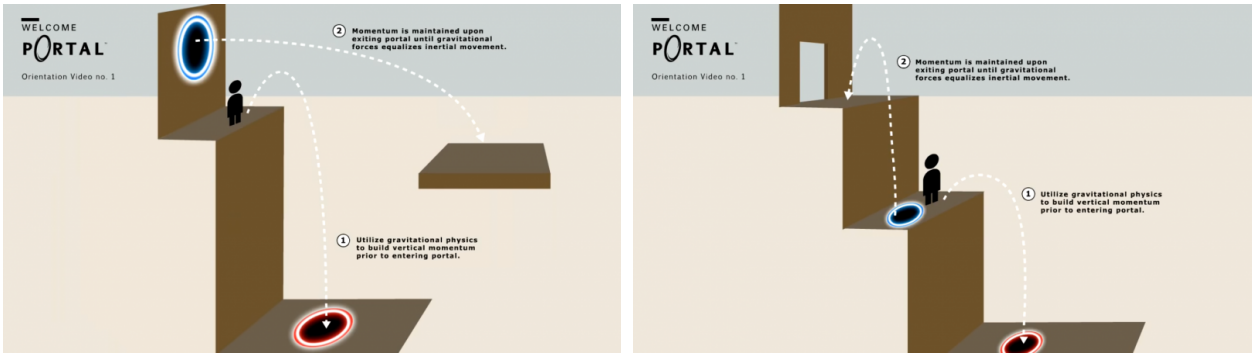
Portal puzzles can also involve significant amount of air travel. Players are expected to move and even shoot portals while airborn. The player can move forward, backward, left or right. Additionally, the player can also pick up/interact with objects, crouch or jump. Finally each of these action is augmented with mouse movement. Often expert players will learn small sequences of these unit actions as a one "macro-action". Figure 2 illustrates 2 common "macro-actions" learned by all the players to solve subpuzzles in a chamber. Ingenuity in portal puzzles come from a really satisfying composition of puzzle elements as well as the player mechanics. Puzzles can sometimes be so difficult that even players who have played hundreds of community made chambers, can take more than an hour [4] to figure out the solution to a difficult puzzle. The referenced video also contains a *custom*, community-made mechanic called "sendificate" which is not present in the original game. The player discovers capabilities of this new mechanic in first 2 introductory sub-puzzles while the third and final subpuzzle is a complex application of concepts introduced in first 2 subpuzzles.

This means that the learned policy/algorithm should be able to identify and adapt to "learning" signals from the environment. The general human strategy to solve a puzzle is to *explore* the chamber as much as possible, then mentally simulate the sequence of actions and seeing what sequence can be beneficial either for further exploration or for reaching/opening exit. This behavior is similar to Monte-Carlo Tree Search by Kocsis and Szepesvári [2006] except that the simulation and execution steps are both very nuanced and expensive. The agent's planning prediction of "move

[2] https://store.steampowered.com/app/620/Portal_2

[3] https://theportalwiki.com/wiki/Mechanics

[4] https://www.youtube.com/watch?v=BGIPzdq8KXM



(a) Fundamental technique to jump over chasms using portals (b) Building vertical height using portals

Figure 2: Demonstration of "flinging" – a fundamental "macro-action" to avoid and jump over obstacles in Portal 2. Image credit: https://theportalwiki.com/wiki/Flinging

forward 10 steps, shoot a blue portal at 45 degree angle" can turn out to be completely wrong simply because the puzzle maker has made puzzle in such a way that no portal-able wall is visible at 45 degree angle after walking for 10 steps.

Creating new puzzles for Portal 2 is also a fairly straightforward process. There is an in-game puzzle editor which can be used by anyone to create and publish a new puzzle. In fact, the puzzle editor is so easy and intuitive that an article by LaBonte [2014] shows high school teachers using this editor to teach science to high school students.

A good RL environment for Portal 2 must run faster than the original game. It must be very easy to create new puzzles for this environment. Last but not the least, it should also be possible to record human players playing a particular chamber for supervision as done in Silver et al. [2016].

3 Related Work

Seminal works in RL have used already existing application programming interfaces (APIs) in various games to develope their agents e.g. Berner et al. [2019] used DOTA2 "bot" API, Fuchs et al. [2021] used Gran Turismo ML API. Other works like Mnih et al. [2015] used an emulator based learning environment by Bellemare et al. [2013] to emulate Arcade games. Other emulator based environments include Nintento emulator based environment by Nichol et al. [2018]. First person shooter environments by Kempka et al. [2016] and Beattie et al. [2016] also exist. These environments also let researchers design their own custom tasks. Although, the flexibility is limited. For example, visual style cannot be changed a lot and custom elements cannot be added without developer-level familiarity with the base games (ZDoom[5] in case of Kempka et al. [2016] and ioquake3[6] in case of Beattie et al. [2016]). Owing to popularity of reinforcement leaning being applied to computer games, game engine developers have also collaborated with researchers to directly bake RL interfaces in the game engines. Unity ML agents by Juliani et al. [2020] and Godot RL agents by Beeching et al. [2021] are 2 prominent examples of this.

The NetHack challenge by Küttler et al. [2020] is similar to TESTCHAMBER since NetHack also requires human-like reasoning (as opposed to pure dexterity requirements as discussed in the introduction). NetHack and TESTCHAMBER both require agents to exhibit "slow thinking" (as discussed in Kahneman [2011]) where the system must carefully reason and plan instead of quickly reacting to the environment (referred as "fast thinking" by Kahneman [2011]). NetHack challenge is however, far simpler than TESTCHAMBER since there is no "perception" required – the currently explored states are directly available to you. As opposed to this, TESTCHAMBER not only requires the agent to generate but also track its position in mental map of the chamber for successful generalization. More details of the observation and states will be discussed further.

4 TESTCHAMBER: Development and Specification

Based on the reviewed literature and the requirements, it was concluded that there are 4 possible ways to develope TESTCHAMBER. At the start of the project, only portal gun was planned to be implemented. However, as the project

[5] https://zdoom.org
[6] https://ioquake3.org

went on, the importance of other elements also became apparent. We first discuss the 4 possible ways and then justify the final choice.

1. **Modding** – Implementing Portal gun and other puzzle elements in a game engine that has already been exposed RL. Examples of these engines are ViZDoom by Kempka et al. [2016], DeepMind Lab by Beattie et al. [2016] or Minecraft (malmo platform) by Johnson et al. [2016].
2. **Scratch-RL** – Implementing the entire game from scratch in a game engine that is RL ready e.g. Unity (with the help of Unity ML agents by Juliani et al. [2020]) or Godot (with the help of Godot RL agents API developed by Beeching et al. [2021]).
3. **Scratch-Full** – Implementing the entire game from scratch in a completely standalone environment (without any game engine).
4. **Portal-Plugin** – Implementing the interface using a plugin for Portal 2's game engine named Source.

4.1 Development

Each approach was systematically explored for 2 weeks studying advantages and disadvantages of the process. The summary of this study is presented below.

Modding approach was the easiest to start with since the RL interface between game engine and the agent has been thoroughly tested. Furthermore, the community of the respective games had already created Portal gun modifications for both the games. Despite this, it was incredibly difficult to work with DeepMind Lab since they had abstracted so many raw engine details behind Lua API. Therefore, only ViZDoom environment with Portal gun modification was tested. With this environment, an agent was trained to test the efficiency and overall fidelity of the training process. Results and specifications for this agent will be discussed in Sec. 5 further. Unfortunately, this approach was not chosen to be the final implementation due to lack of ability to effectively record human plays and due to lack of a good, intuitive, easy-to-use level editor for creating new levels.

Scratch-RL approach was more challenging than **Modding** approach but even here passionate game developers have open sourced implementation of Portals in both the game engines under consideration. However, these implementations are barely faster than the original game itself which is a dealbreaker. Optimizing these implementations is possible but may require more time than was allotted for this project.

Similar story goes for the **Scratch-Full** approach. Implementing from scratch, more focus was kept on making the behavior of the player as close to real game as possible which meant that agents trained could be transferred to original game for comparison with humans. Unfortunately, neither of the 2 implementations were significantly faster than the original game. Furthermore, it would also be required to build a custom level editor and custom functionality to record human gameplay in this engine for this approach to be useful. Again, due to time constraint, this approach was not pursued further.

Finally, **Portal-Plugin** approach was chosen to be the appropriate one. This is majorly thanks to the community developed `SourceAutoRecord` plugin[7]. `SourceAutoRecord` is used primarily for recording the "demo" and the time for "speedrunning" the game. "Speedrunning" is the act of completing the game (here, Portal 2) as fast as possible. Speedrunning requires accurate measurement of the completion time of each individual level of the game which is what is primarily accomplished by `SourceAutoRecord` plugin. Additionally, `SourceAutoRecord` also gives an interface for "Tool Assisted Speedruns". Tool assisted speedrun (TAS) is simply a sequence of commands to the game engines that precisely controls the movement of the player. The TAS player (implemented by the developers of `SourceAutoRecord` plugin) reads the sequence of commands from a file on the disk and executes the movement instructions at precise game engine cycles as specified in the file. Passionate human players will often spend *weeks* perfectly specifying the inputs in the TAS file optimizing the number of ticks (another name for game engine cycle) required to finish a particular level. Since TAS player is not limited by the human response times and can *always perfectly* follow the instructions in the TAS file, TAS is often *significantly* faster than a human speedrunner[8].

The final implementation of the TESTCHAMBER builds on top of TAS interface implemented in `SourceAutoRecord`. More detailed information is available here : `https://wiki.portal2.sr/TASsing`. Typical TAS workflow makes the `SourceAutoRecord` read a list of TAS *framebulks* (a unit of movement specification in `SourceAutoRecord`, please refer to the aforementioned link for more info) from a text file. This work modifies the TAS workflow so that it accepts a single *framebulk* over the network. This enables the Gym interface to predictably control the in-game TAS player and observe current game state.

[7] Primarily maintained by Matthew Lugg (mlugg). Source: `https://github.com/p2sr/SourceAutoRecord`

[8] Portal 2 co-op TAS (`https://www.youtube.com/watch?v=tbKhToSKM64`) is more than 10 *minutes* faster than human speedrun (`https://www.youtube.com/watch?v=U86yDlGEYHk`)

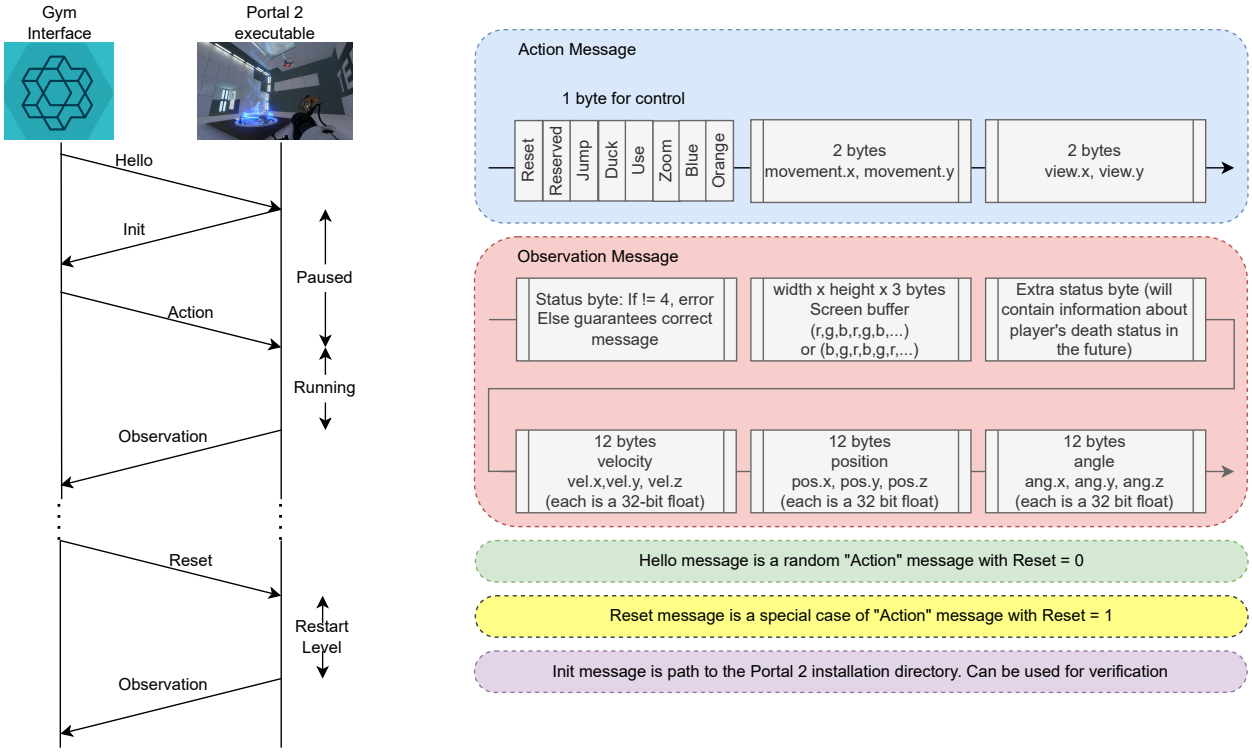


Figure 3: Protocol used by the Gym interface to communicate with the Portal 2 executable

4.2 Specification

Figure 3 demonstrates the protocol that Gym interface and Portal 2 communicate on. To setup Portal 2 for RL training, users must make sure that they own Portal 2 via Steam. Then they must install the modified version of `SourceAutoRecord` from my repository : https://github.com/ashutoshbsathe/SourceAutoRecord (instructions for installation are same as original `SourceAutoRecord`). After that, following must be added to the "Launch Options" on Steam :

```
-vulkan -high -novid -sw -w 180 -h 180 -condebug +map rl_challenge_1 +plugin_load sar
+sar_tas_server 1 +sar_tas_debug 1 +sar_tas_playback_rate 100
+hud_quickinfo 0 +sar_quickhud_mode 1 +sar_quickhud_size 4
```

Map name `rl_challenge_1` can be modified to the map of user's requirements. Width and height are currently recommended at 180 as going any lower crashes the game on Linux and going any above limits the processing. Also notice `sar_tas_playback_rate 100` argument, this speeds up the playback of TAS i.e. playback of game when we issue a command. Depending on the host computer performance, this can result in a speedup of about 2x-100x[9]. Furthermore, the server-client architecture as shown in the Fig. 3 enables the model to be trained in isolation (on a server without any display) while the game is running on a fast enough computer with display connected. Additionally, `SourceAutoRecord` also provides toolkits necessary for automatically recording humans while they are solving a puzzle. This recording is highly efficient since it stores only the commands to the game engine rather than recording the entire screen. Such recorded demos (extension `.dem` files) can also be loaded in the game to generate a dataset for supervised training/behavior cloning. Details of this conversion are omitted for brevity. Please refer to the source repository for more info.

Fig. 3 also highlights that Portal 2 server will run in isolation independently while the client (Gym interface or any other interface) can be implemented in any language as per users' requirements. A reference Gym interface implementation is available at https://github.com/ashutoshbsathe/TestChamber. RL training specific technical details are presented below :

[9] VSync must be turned off

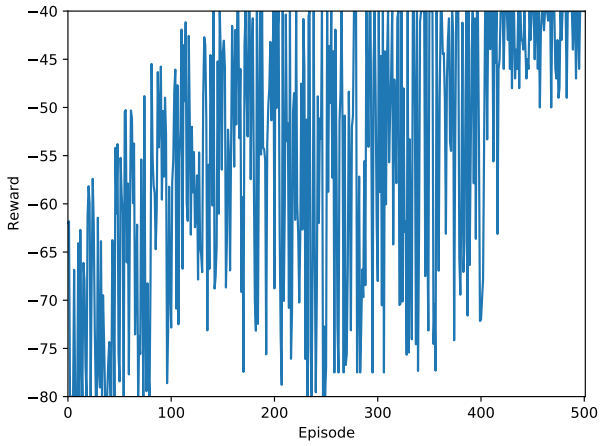


Figure 4: Reward as a function of episode in ViZDoom + Portal modification

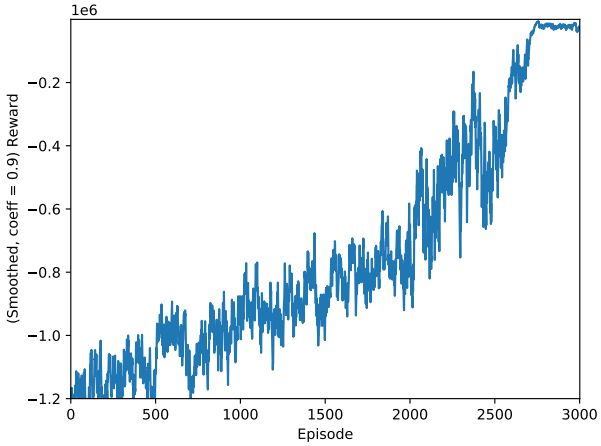


Figure 5: Reward as a function of episode in bandit-like formulation

4.2.1 Action

In `SourceAutoRecord` terms, each actions is simply a *framebulk*. Since it would be cumbersome to let the model control *number of ticks* the framebulk runs *as well as* the framebulk itself, it is assumed that each framebulk runs for exactly 4 in-game ticks. Each framebulk is serialized as shown in Fig. 3. Furthermore, movement and view (x, y) values are quantized between 0-255 instead of them being purely continuous. Each action therefore consists a combination of :

1. Button states (j, d, u, z, b, o) for each of the (jump, duck, use, zoom, blue portal, orange portal) buttons. Here, $d = 1$ *presses* the “duck" button and $d = 0$ *releases* the “duck" button. Between $d = 1$ and $d = 0$, the button will be constantly pressed
2. Movement pair (m_x, m_y) where each m_x, m_y is one of the 0-255 numbers. Each of these is scaled to a floating point number between -1 (corresponds to 0) to 1 (corresponds to 255) as required by the framebulk specifications
3. View pair (v_x, v_y) exactly similar to movement pair

4.2.2 Observation

Observation consists of information that is available to human players. After an action is performed, the game pauses and sends the observation back to the client. The game keeps waiting till any new action is received. Each of the observation contains a first flag byte whose value is 4. This indicates that the incoming data is perfect. If there is any other value, client must detect error and handle it (or simply reset the level). In a perfect observation, server will then send $w \times h \times 3$ bytes as shown in Fig. 3. Furthermore, the server will also send player’s velocity, angle and position as floating-point triplets back to the client. This information is often used by speedrunners to accurately aim or *slow down* at appropriate places.

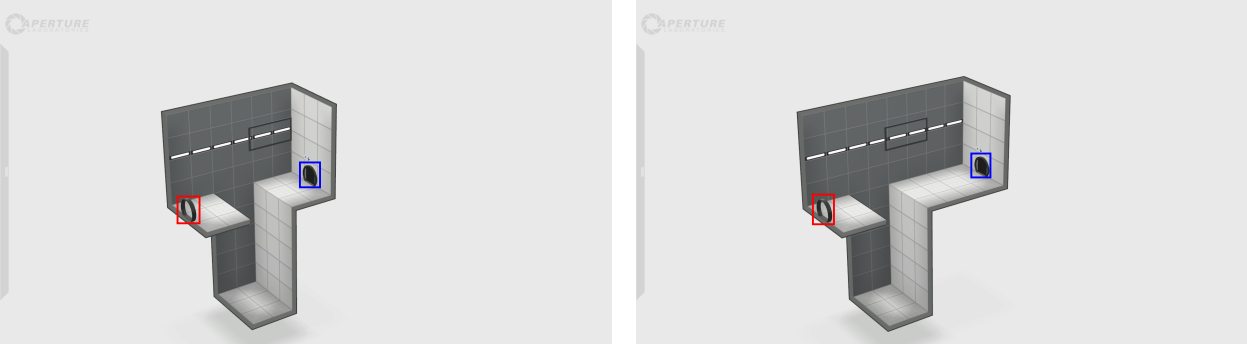
4.2.3 Reward

Reward specification is left to user. As will be shown via experiments, there is no single clear, easy or correct way to define reward in this environment.

5 Experiments

This section addresses feedback on mid-term presentation as well as presents new results on the latest environment.

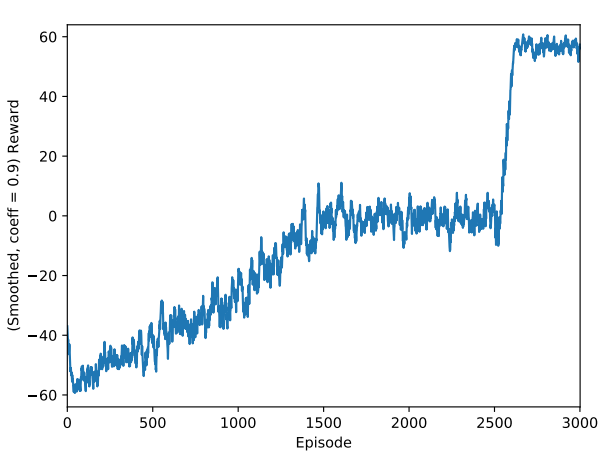
Regarding mid-term feedback. In mid-term presentation, 2 agents were presented. The first one was a baseline with ViZDoom environment and portal gun modification. The goal of this agent was to reach the end of the level. The states were represented using a ConvNEt that read direct pixel values from the interface provided by ViZDoom. This ConvNet was pretrained with a decoder to reproduce the image first (in order to build some notion of similarity at least)



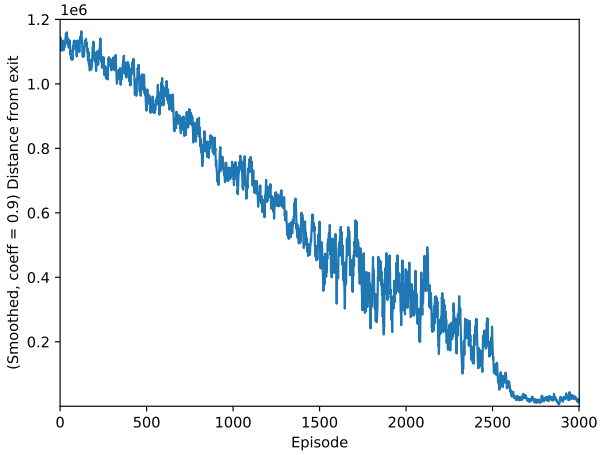
(a) Map `RL_Challenge_0`

(b) Map `RL_Challenge_0.5`

Figure 6: Chambers used for experimentation. The red doors indicate entrance doors and blue doors indicate exit doors. The agent needs to put 2 portals – one near red door and another near blue door and simply needs to walk through them in order to reach the exit i.e. the blue door.



(a) Reward vs episode in `RL_Challenge_0` map (continuous control)



(b) Distance to exit vs episode in `RL_Challenge_0` map (continuous control)

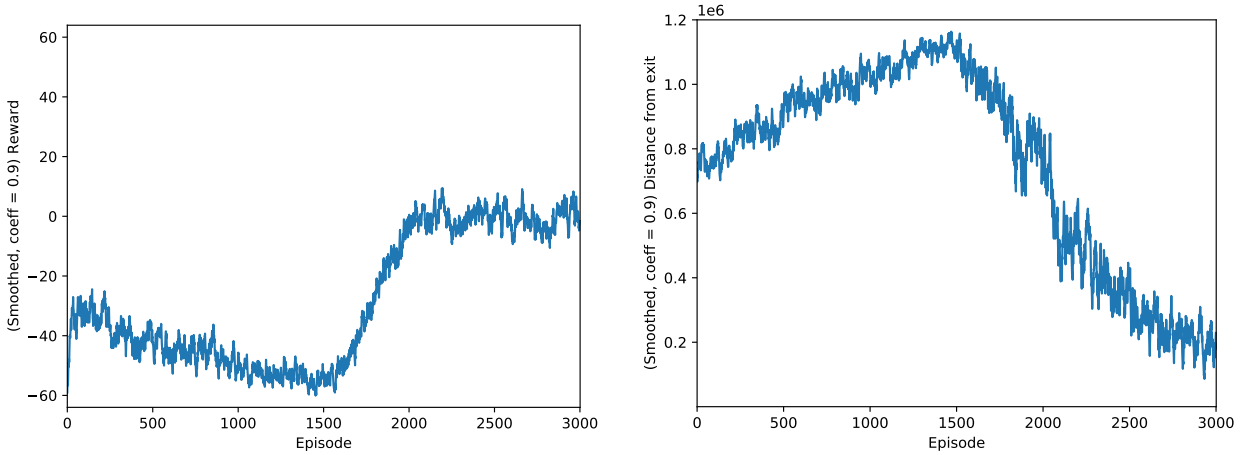
Figure 7: Training from scratch on `RL_Challenge_0` map

like an autoencoder. The actions were similar definitions in Sec. 4.2.1 and the reward was 0 for goal states and -1 for all states except goal state. Fig. 4 shows the plot for reward vs episode for this experiment. The agent was able to complete the chamber in $\sim$ 10-12 seconds where the agent interacted for about 4 times per second with the environment. Despite the decent performance, this approach was not pursued (as discussed previously) due to challenges in level editor and mapping.

The mid-term presentation also presented a bandit-like formulation where the agent was trained to directly predict a sequence of 64 framebulks. This is bandit-like since the sequence of 64 framebulks can be considered 1 arm of the bandit. The reward function was simply negative of the distance between the final position of the player and the exit door. Reward vs episode plot for this experiment is shown in Fig. 5. The map used for this experiment is similar to as shown in Fig. 6a.

Next, results for newer experiments with continuous control (as described in Sec. 4.2) are presented. 2 maps (along with their expected solutions) as shown in Fig. 6 are considered. Only the reward function that gave the best results is presented for brevity. For both the maps, the reward function at step i is defined as follows :

1. $+1$ reward if the agent is closer to the exit door than it was in steps $0, \ldots, i-1$
2. -1 reward if the agent is farther from the exit door than it was in step $i-1$
3. 0 reward if none of the above



(a) Reward vs episode in `RL_Challenge_0.5` map (continuous control)

(b) Distance to exit vs episode in `RL_Challenge_0.5` map (continuous control)

Figure 8: Finetuning the model trained on `RL_Challenge_0` map to `RL_Challenge_0.5` map

Maximum episode length is kept at 64 to account for modest compute availability. With this, the agent was trained using PPO (Proximal Policy Optimization due Schulman et al. [2017]) implementation from RLLib library by Liang et al. [2018] for 3000 epochs. More details about hyperparameters and model architecture can be found on the code repository[10].

Training plots for the map `RL_Challenge_0` are shown in Fig. 7. It can be observed that reward slowly increases from -64 towards zero. The very first agents were getting stuck at the entry door and were being pushed away due to game physics hence the high negative reward. In the middle of the training, the reward settles at around 0 since the agent keeps falling in the hole that it cannot climb out of and hence keeps cancelling all the accrued positive reward to get 0 episode reward. This is also evidenced by the high amount of noise observed in distance-vs-episode plot (Fig. 7b). The agent will also often shoot portal on the pit floor which could sometime *fling* the agent towards the door. It should also be noted that at this stage in the training, the agent is shooting portals incredibly randomly but this behavior suddenly stops late training and the agent is able to shoot correct pair of portals and is able to reach the door. Please view the accompanying video for more clarity.

Curiously, the agent developes a biased behavior where it keeps using the blue portal as the entry portal and orange portal as the exit portal. When these buttons are swapped, the trained agent catastrophically fails to shoot correct portals. This behavior is strange and suggests that agent has simply memorized the sequence of actions.

To further explore this, the exit door was moved farther back in map `RL_Challenge_0.5` (Fig. 6b). Functionally, maps `RL_Challenge_0` and `RL_Challenge_0.5` are *exactly* the same and so the expectation was that the agent trained on `RL_Challenge_0` will flawlessly transfer to `RL_Challenge_0.5`. Unfortunately, the results indicated that the agent just kept falling into the pit when the door was moved back. This can potentially happen because visually `RL_Challenge_0.5` will look different due to door being farther.

So further experiments were done to see whether fine-tuning can quickly increase the performance. Training plots for this experiment are shown in Fig. 8. The results, however, suggest that the fine-tuning had a negative impact on the agent where in initial stages of training agent became *worse*. This suggests behavior like catastrophic forgetting might be occurring which causes the agent to first become *worse*. In the later stages, the agent improves, but not a lot. The training was stopped after 3000 episodes after which the best model was still falling in the pit and not reaching anywhere close to the exit.

6 Conclusion

This report proposed TESTCHAMBER – a novel and challenging environment for reinforcement learning. The proposed environment is based on a puzzle game “Portal 2" where the goal of the player is to reach the exit door of a test-chamber. Key highlights of the proposed environments are as follows :

[10] https://github.com/ashutoshbsathe/TestChamber

1. Server-client interface – The RL interface (as a client) can be implemented in any language as long is follows the protocol (described in Fig. 3)
2. Faster execution of the game engine itself – The Portal 2 game is fairly old and many modern computers can run the game at extremely high framerates. TESTCHAMBER takes advantage of this to speed up the game engine itself for faster training
3. Isolated display nodes – Another benefit of server-client architecture. The game can simulated at extremely high speeds using a machine connected to display somewhere while the main training can happen on a far more powerful server that may or may not have display connections
4. Generating datasets from human playthroughs – TESTCHAMBER can extract training data from publicly available demo files of human players (such as https://board.portal2.sr) which can be useful for supervised learning or behavioral cloning

The report further presented arguments suggesting that solving "Portal 2" puzzles requires "higher-level/slow" thinking beyond just dexterity which has been the focus of large body of previous work in RL. Experimental results demonstrated challenges and behaviors observed during solving TESTCHAMBER using existing RL algorithms. We hope that this work motivates further research on better RL algorithms that generalize better and able to do interpretable and explainable reasoning. We conclude this report by suggesting some test-chambers for further research.

- No elements collection – https://steamcommunity.com/workshop/filedetails/?id=129519365 – set of maps that only use *flinging* mechanic and nothing else. The agent can be trained on first 18 maps and tested on maps 19 and 20.
- Custom visuals – Many great puzzles by the talented puzzle maker "Demon Arisen" (https://steamcommunity.com/id/demonarisen/myworkshopfiles/?appid=620) are available in various visual styles – a challenge would be to train an agent that learns to solve all or a subset of these puzzles in one visual style and test agent's generalization on different visual styles.
- Sendificate series – https://steamcommunity.com/workshop/filedetails/?id=291653034 – set of 4 maps where a completely new game mechanic called "sendificate" is introduced. Agent can be trained on first 3 maps only and should be able to solve the last map on its own.

This list is by no means an exhaustive. Readers are encouraged to explore the Steam workshop page themselves to find and recommend more challenges. The tools developed during this project are linked below :

Gym interface and tools – https://github.com/ashutoshbsathe/TestChamber

Custom version of `SourceAutoRecord` – https://github.com/ashutoshbsathe/SourceAutoRecord

Acknowledgements

Thanks to the `SourceAutoRecord` development team for continued and insightful support through the project. Special thanks to Matthew Lugg for developing a tool for converting demo videos to TAS files (https://github.com/mlugg/dem2tas).

References

B. Baker, I. Kanitscheider, T. Markov, Y. Wu, G. Powell, B. McGrew, and I. Mordatch. Emergent tool use from multi-agent autocurricula. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=SkxpxJBKwS.

C. Beattie, J. Z. Leibo, D. Teplyashin, T. Ward, M. Wainwright, H. Küttler, A. Lefrancq, S. Green, V. Valdés, A. Sadik, J. Schrittwieser, K. Anderson, S. York, M. Cant, A. Cain, A. Bolton, S. Gaffney, H. King, D. Hassabis, S. Legg, and S. Petersen. Deepmind lab, 2016.

E. Beeching, J. Debangoye, O. Simonin, and C. Wolf. Godot reinforcement learning agents, 2021.

M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, Jun 2013. ISSN 1076-9757. doi:10.1613/jair.3912. URL http://dx.doi.org/10.1613/jair.3912.

C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, R. Józefowicz, S. Gray, C. Olsson, J. Pachocki, M. Petrov, H. P. de Oliveira Pinto, J. Raiman, T. Salimans, J. Schlatter, J. Schneider, S. Sidor, I. Sutskever, J. Tang, F. Wolski, and S. Zhang. Dota 2 with large scale deep reinforcement learning. *CoRR*, abs/1912.06680, 2019. URL http://arxiv.org/abs/1912.06680.

G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. Openai gym, 2016. URL https://arxiv.org/abs/1606.01540.

F. Fuchs, Y. Song, E. Kaufmann, D. Scaramuzza, and P. Durr. Super-human performance in gran turismo sport using deep reinforcement learning. *IEEE Robotics and Automation Letters*, 6(3):4257–4264, Jul 2021. ISSN 2377-3774. doi:10.1109/lra.2021.3064284. URL http://dx.doi.org/10.1109/LRA.2021.3064284.

M. Johnson, K. Hofmann, T. Hutton, and D. Bignell. The malmo platform for artificial intelligence experimentation. In *IJCAI*, 2016.

A. Juliani, V.-P. Berges, E. Teng, A. Cohen, J. Harper, C. Elion, C. Goy, Y. Gao, H. Henry, M. Mattar, and D. Lange. Unity: A general platform for intelligent agents, 2020.

D. Kahneman. *Thinking, fast and slow*. Farrar, Straus and Giroux, New York, 2011. ISBN 9780374275631 0374275637. URL https://www.amazon.de/Thinking-Fast-Slow-Daniel-Kahneman/dp/0374275637/ref=wl_it_dp_o_pdT1_nS_nC?ie=UTF8&colid=151193SNGKJT9&coliid=I3OCESLZCVDFL7.

M. Kempka, M. Wydmuch, G. Runc, J. Toczek, and W. Jaśkowski. Vizdoom: A doom-based ai research platform for visual reinforcement learning, 2016.

L. Kocsis and C. Szepesvári. Bandit based monte-carlo planning. In J. Fürnkranz, T. Scheffer, and M. Spiliopoulou, editors, *Machine Learning: ECML 2006*, pages 282–293, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg. ISBN 978-3-540-46056-5.

H. Küttler, N. Nardelli, A. Miller, R. Raileanu, M. Selvatici, E. Grefenstette, and T. Rocktäschel. The nethack learning environment. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 7671–7684. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper/2020/file/569ff987c643b4bedf504efda8f786c2-Paper.pdf.

D. LaBonte. Game-based STEM introduction. https://www.edutopia.org/blog/reinventing-science-fair-portal-2-don-labonte, 2014.

E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, and I. Stoica. Rllib: Abstractions for distributed reinforcement learning. In *International Conference on Machine Learning*, pages 3053–3062. PMLR, 2018.

V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, Feb 2015. ISSN 1476-4687. doi:10.1038/nature14236. URL https://doi.org/10.1038/nature14236.

A. Nichol, V. Pfau, C. Hesse, O. Klimov, and J. Schulman. Gotta learn fast: A new benchmark for generalization in rl. *arXiv preprint arXiv:1804.03720*, 2018.

T. Pearce and J. Zhu. Counter-strike deathmatch with large-scale behavioural cloning, 2021.

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, Jan 2016. ISSN 1476-4687. doi:10.1038/nature16961. URL https://doi.org/10.1038/nature16961.

O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, and D. Silver. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, Nov 2019. ISSN 1476-4687. doi:10.1038/s41586-019-1724-z. URL https://doi.org/10.1038/s41586-019-1724-z.